# Informe SaludUSM

## Sección 4.1 Diseño del modelo documental

En la siguiente sección, se muestra cómo se diseña el modelo documental de SaludUSM para migrar la base de datos desde MySQL a MongoDB. Para realizar este diseño, primero es necesario entender qué es MongoDB y cómo organiza la información mediante documentos y colecciones.

MongoDB es una base de datos no relacional (NoSQL) basada en documentos. A diferencia de las bases de datos relacionales con las que se ha trabajado anteriormente, organiza la información mediante documentos agrupados en colecciones.

¿Qué son los documentos?

Los documentos son registros de una base de datos documental que permiten almacenar información sobre un objeto mediante pares campo-valor. En MongoDB, los documentos se almacenan internamente en formato BSON. Para este informe, se utilizará JSON para representar ejemplos de documentos y, posteriormente, para entregar las exportaciones solicitadas por el enunciado. Un ejemplo de un documento representado en JSON, basado en un centro médico de **salud_usm**, es el siguiente:

```
{

"_id": 1,
"nombre": "SaludUSM San Joaquín",
"comuna": "San Joaquín",
"region": "RM"
}
```

En el ejemplo anterior, se observa que un documento cumple una función similar a una fila de una tabla relacional, ya que representa un registro con sus respectivos datos. Los documentos que representan entidades del mismo tipo, como los centros médicos, se pueden agrupar en una colección.

¿Qué son las colecciones?

Las colecciones son conjuntos de documentos y cumplen una función similar a las tablas de una base de datos relacional. En MongoDB, los documentos de una colección no están obligados a tener los mismos campos ni tipos de datos. Sin embargo, para SaludUSM se definirá una estructura común que permita organizar y consultar la información de manera consistente.

No todas las tablas del modelo relacional deben convertirse directamente en una colección. Esta decisión depende de cómo se consultará la información y de las relaciones entre los datos. Para diseñar las colecciones de **salud_usm**, se utilizará una descripción de sus campos y tipos de datos, como se muestra en el siguiente ejemplo de la colección Centros: Centros: { _id: Integer, nombre: string, comuna: string, region: string } Este formato se parece a la representación de un documento en JSON, pero se utiliza para describir su estructura. En lugar de mostrar valores concretos, indica el tipo de dato de cada campo y, al principio, señala el nombre de la colección.

Se mantienen `Pacientes` y `Citas` como colecciones separadas porque las citas se consultan también por médico, fecha y centro. Cada cita almacena la referencia al paciente, evitando repetir todos sus antecedentes en cada reserva.

Se mantienen `Citas` y `Atenciones` separadas porque una cita puede estar reservada, cancelada o pendiente de atención. El documento de atención se incorpora cuando existe información clínica que registrar y conserva la referencia a su cita.

Las especialidades y centros asignados a cada médico se representan mediante arreglos de identificadores. Esto permite consultar sus asignaciones desde el documento del médico, mientras que los nombres y otros datos descriptivos permanecen en sus respectivas colecciones.

Basándose en esta información y en el modelo normalizado de la Tarea 2, se proponen las siguientes colecciones para SaludUSM. Para definirlas, se consideran principalmente los datos necesarios para consultar el *Comprobante de Reserva de Hora* y la *Ficha de Atención*.

```
Comprobantes: {
    _id: Integer,
    generado_en: Date,
    cita: {
        id: Integer,
        fecha: Date,
        hora: String,
        estado: {
            id: Integer,
            nombre: String
        },
        paciente: {
            rut: String,
            nombre: String,
            telefono: String | Null,
            prevision: {
                id: Integer,
                tipo: String
            }
        } | Null,
        medico: {
            rut: String,
            nombre: String,
            especialidad: {
                id: Integer,
                nombre: String
            }
        },
        centro: {
            id: Integer,
            nombre: String,
            comuna: String,
            region: String
        }
```

```
    }
}
```

Cada comprobante utiliza como `_id` el identificador de su cita. El campo `generado_en` registra cuándo se generó el documento. La hora se representa mediante una cadena con formato `HH:MM:SS`. El paciente puede ser `null` porque el origen permite conservar citas cuya referencia al paciente fue eliminada.

```
Fichas: {
    _id: Integer,
    generado_en: Date,
    cita: {
        id: Integer,
        fecha: Date,
        hora: String,
        centro: {
            id: Integer,
            nombre: String
        },
        paciente: {
            rut: String,
            nombre: String,
            nacimiento: Date,
            sexo: String
        } | Null,
        medico: {
            rut: String,
            nombre: String,
            email: String | Null,
            especialidad: {
                id: Integer,
                nombre: String
            }
        },
        atencion: {
            id: Integer,
            motivo: String,
            observacion: String,
            diagnosticos: [
                {
                    codigo: String,
                    descripcion: String
                }
            ],
            recetas: [
                {
                    receta_id: Integer,
                    medicamento: String,
                    dosis: String,
                    dias: Integer
                }
            ]
```

```
        }
    }
}
```

Cada ficha utiliza como `_id` el identificador de su atención. Dentro de ella se reúnen los antecedentes de la cita, el paciente, el médico y el centro. Los diagnósticos y las recetas se representan mediante arreglos porque una atención puede contener varios de ellos.

En `Atenciones`, los diagnósticos se almacenan como referencias a su catálogo. En `Fichas`, se copian sus códigos y descripciones para presentar conjuntamente los antecedentes de la atención.

```
Citas: {
    _id: Integer,
    fecha: Date,
    hora: String,
    estado_id: Integer,
    paciente_rut: String | Null,
    medico_rut: String,
    especialidad_id: Integer,
    centro_id: Integer
}

Pacientes: {
    _id: String,
    nombre: String,
    telefono: String,
    nacimiento: Date,
    sexo: String,
    prevision_id: Integer,
    email: String | Null,
    contrasena_hash: String | Null
}

Medicos: {
    _id: String,
    nombre: String,
    email: String,
    centro_ids: Array<Integer>,
    especialidad_ids: Array<Integer>,
    contrasena_hash: String | Null
}

Centros: {
    _id: Integer,
    nombre: String,
    comuna: String,
    region: String,
    medicos_rut: Array<String>
}

Estados: {
```

```
    _id: Integer,
    nombre: String
}

Previsiones: {
    _id: Integer,
    tipo: String
}

Especialidades: {
    _id: Integer,
    nombre: String
}

Atenciones: {
    _id: Integer,
    cita_id: Integer,
    motivo: String,
    observacion: String,
    diagnosticos_id: Array<String>,
    recetas: [
        {
            receta_id: Integer,
            medicamento: String,
            dosis: String,
            dias: Integer
        }
    ]
}

Diagnosticos: {
    _id: String,
    descripcion: String
}
```

Las estructuras anteriores servirán como base para construir los documentos de cada colección de **salud_usm**. Las colecciones propuestas no corresponden exactamente a todas las tablas obtenidas al normalizar las vistas hasta 3FN, ya que el modelo documental permite organizar los datos mediante documentos embebidos y referencias, según cómo se utilizará la información.

Para este caso, se mantienen previsión, especialidad y estado de cita como colecciones de catálogo, de acuerdo con lo solicitado en el enunciado. Los documentos que necesiten estos datos almacenarán sus identificadores como referencias. Al insertar o modificar información, se comprobará que dichos identificadores existan en los catálogos correspondientes.

Diagnósticos se mantiene como una colección independiente porque un mismo diagnóstico puede estar asociado a distintas atenciones. Cada atención almacenará un arreglo con los códigos de sus diagnósticos, mientras que sus descripciones se conservarán en la colección Diagnósticos.

Por otro lado, las recetas se almacenarán como un arreglo de documentos embebidos dentro de cada atención. Cada receta contendrá el medicamento, la dosis y los días de tratamiento. Esta decisión permite

consultar las indicaciones entregadas junto con la atención a la que pertenecen.

`Comprobantes` y `Fichas` almacenan copias de los datos necesarios para presentar las vistas de usuario. En las operaciones implementadas, la reserva crea su comprobante, la cancelación actualiza el estado del comprobante y la eliminación de una receta actualiza también la ficha correspondiente.

Las modificaciones generales de nombres y otros antecedentes duplicados requerirían actualizar o regenerar las copias afectadas. Esta responsabilidad se debe considerar al ampliar el sistema.

Las relaciones que en el modelo relacional se representaban mediante claves foráneas se expresarán mediante referencias entre documentos. Una referencia consiste en almacenar el identificador de otro documento. Por ejemplo, una cita contendrá el RUT del paciente y del médico, además de los identificadores del centro, la especialidad y el estado de la cita.

Estas referencias permiten identificar los documentos relacionados, pero no comprueban automáticamente que existan. Por ello, el código que inserte o modifique las citas deberá verificar que los datos referenciados sean válidos y que el médico pertenezca al centro y tenga asignada la especialidad indicada.

La relación entre pacientes y citas es de uno a muchos, porque un paciente puede tener varias citas y cada cita corresponde a un paciente. Se mantendrán ambas colecciones separadas y cada cita almacenará el RUT del paciente como referencia.

La relación entre citas y atenciones permite que una cita tenga ninguna o una atención, mientras que cada atención corresponde a una cita. Se mantendrán como colecciones separadas y cada atención almacenará el identificador de su cita. Se deberá impedir que una misma cita tenga más de una atención.

Las relaciones médico–especialidad y médico–centro son de muchos a muchos. Un médico puede tener varias especialidades y trabajar en varios centros, mientras que cada especialidad y cada centro pueden estar asociados a distintos médicos. Estas relaciones se representarán mediante los arreglos de identificadores incluidos en los documentos de los médicos.

La colección Centros también almacenará los RUT de sus médicos. Como esta relación estará representada tanto en Centros como en Médicos, cualquier cambio de asignación deberá actualizar ambos documentos para mantener la información consistente.

Este diseño establece cómo se organizarán los datos de SaludUSM en MongoDB y cómo se representarán sus relaciones. A partir de estas estructuras, se prepararán los documentos y se realizará la migración de los datos desde MySQL mediante Python.

## Ejemplos de documentos

Los siguientes documentos ilustran la estructura propuesta para cada colección. Los valores son representativos y mantienen las referencias entre documentos.

**Centros**

```
{
  "_id": 1,
  "nombre": "SaludUSM San Joaquín",
  "comuna": "San Joaquín",
```

```
  "region": "RM",
  "medicos_rut": ["00000000-1", "12345678-9"]
}
```


## Especialidades


```
{
  "_id": 1,
  "nombre": "Cardiología"
}
```


## Previsiones


```
{
  "_id": 2,
  "tipo": "Isapre"
}
```


## Estados


```
{
  "_id": 2,
  "nombre": "Atendida"
}
```


## Diagnosticos


```
{
  "_id": "A09",
  "descripcion": "Gastroenteritis de presunto origen infeccioso"
}
```


## Pacientes


```
{
  "_id": "00000000-0",
  "nombre": "Martina Díaz",
  "telefono": "912345670",
  "nacimiento": "1995-01-01",
  "sexo": "F",
  "prevision_id": 2,
  "email": "paciente.prueba@example.test",
```

```
  "contrasena_hash": null
}
```

## Medicos

```
{
  "_id": "00000000-1",
  "nombre": "Dr. Andrés Vidal",
  "email": "medicogenerico@gmail.com",
  "contrasena_hash": null,
  "centro_ids": [1],
  "especialidad_ids": [1]
}
```

## Citas

```
{
  "_id": 1,
  "fecha": "2026-09-21",
  "hora": "09:00:00",
  "estado_id": 2,
  "paciente_rut": "00000000-0",
  "medico_rut": "00000000-1",
  "especialidad_id": 1,
  "centro_id": 1
}
```

## Atenciones

```
{
  "_id": 1,
  "cita_id": 1,
  "motivo": "Fiebre y congestión nasal",
  "observacion": "Rinofaringitis; presión arterial elevada en control.",
  "diagnosticos_id": ["A09"],
  "recetas": [
    {
      "receta_id": 1,
      "medicamento": "Paracetamol 500 mg",
      "dosis": "1 comprimido cada 8 horas",
      "dias": 5
    }
  ]
}
```

## Comprobantes


```
{
  "_id": 1,
  "generado_en": "2026-09-21",
  "cita": {
    "id": 1,
    "fecha": "2026-09-21",
    "hora": "09:00:00",
    "estado": {
      "id": 2,
      "nombre": "Atendida"
    },
    "paciente": {
      "rut": "00000000-0",
      "nombre": "Martina Díaz",
      "telefono": "912345670",
      "prevision": {
        "id": 2,
        "tipo": "Isapre"
      }
    },
    "medico": {
      "rut": "00000000-1",
      "nombre": "Dr. Andrés Vidal",
      "especialidad": {
        "id": 1,
        "nombre": "Cardiología"
      }
    },
    "centro": {
      "id": 1,
      "nombre": "SaludUSM San Joaquín",
      "comuna": "San Joaquín",
      "region": "RM"
    }
  }
}
```


## Fichas


```
{
  "_id": 1,
  "generado_en": "2026-09-21",
  "cita": {
    "id": 1,
    "fecha": "2026-09-21",
    "hora": "09:00:00",
    "centro": {
      "id": 1,
```

```
      "nombre": "SaludUSM San Joaquín"
    },
    "paciente": {
      "rut": "00000000-0",
      "nombre": "Martina Díaz",
      "nacimiento": "1995-01-01",
      "sexo": "F"
    },
    "medico": {
      "rut": "00000000-1",
      "nombre": "Dr. Andrés Vidal",
      "email": "medicogenerico@gmail.com",
      "especialidad": {
        "id": 1,
        "nombre": "Cardiología"
      }
    },
    "atencion": {
      "id": 1,
      "motivo": "Fiebre y congestión nasal",
      "observacion": "Rinofaringitis; presión arterial elevada en control.",
      "diagnosticos": [
        {
          "codigo": "A09",
          "descripcion": "Gastroenteritis de presunto origen infeccioso"
        }
      ],
      "recetas": [
        {
          "receta_id": 1,
          "medicamento": "Paracetamol 500 mg",
          "dosis": "1 comprimido cada 8 horas",
          "dias": 5
        }
      ]
    }
  }
}
```

# Sección 4.5 Análisis comparativo: SQL vs. NoSQL

El sistema SaludUSM fue implementado inicialmente mediante tablas relacionales normalizadas hasta tercera forma normal (3FN). En esta tarea, sus datos se migraron desde MySQL hacia MongoDB mediante Python. A continuación, se comparan ambas implementaciones considerando su estructura, las relaciones entre los datos, las reglas de negocio y las consultas necesarias para el funcionamiento del sistema.

## 4.5.1 Modelo de datos y esquema

En MySQL, la información se organiza en tablas cuyas columnas tienen tipos de datos y restricciones definidos. Cada fila representa un registro. Por ejemplo, la tabla `atencion` almacena el número de atención, el motivo, las observaciones y el número de la cita correspondiente.

Para obtener una atención con todos sus antecedentes, se relacionan varias tablas. Los medicamentos se almacenan en `receta`, mientras que los diagnósticos se encuentran en `diagnostico` y se vinculan con las atenciones mediante `atencion_diagnostico`. Esta organización permite mantener separados los datos de cada entidad.

En MongoDB, la información se organiza en colecciones de documentos BSON. En nuestra implementación, un documento de `Atenciones` contiene su identificador `_id`, la referencia `cita_id`, el motivo, las observaciones, un arreglo `diagnosticos_id` y un arreglo `recetas`. Cada elemento de `recetas` incluye el identificador de la receta, el medicamento, la dosis y los días de tratamiento.

Por lo tanto, las recetas se obtienen junto con la atención que las contiene. Los diagnósticos mantienen una colección independiente, ya que sus códigos y descripciones pueden ser utilizados por distintas atenciones. El modelo combina documentos embebidos con referencias según la relación y la forma de consultar los datos.

## 4.5.2 Normalización y redundancia

La normalización a 3FN utilizada en la Tarea 2 reduce la repetición de los datos descriptivos. Por ejemplo, el nombre de un médico se almacena en `medico`, mientras que las citas lo identifican mediante su RUT. Si cambia su nombre, basta con actualizar ese registro para que las consultas que realizan los cruces correspondientes obtengan el valor actualizado.

En MongoDB se mantuvieron colecciones principales como `Medicos`, `Centros`, `Citas` y `Atenciones`. Además, se crearon `Comprobantes` y `Fichas`, que almacenan copias de información necesaria para presentar las vistas de usuario. Estas copias incluyen nombres de médicos, antecedentes del paciente y datos del centro.

Esta decisión permite recuperar información relacionada desde un mismo documento, pero aumenta la redundancia. Si cambia el nombre de un médico o de un centro, actualizar su colección principal no modifica automáticamente los valores copiados en comprobantes y fichas. Si estas copias deben mostrar los datos actuales, la aplicación debe actualizarlas o regenerarlas. Si se desean conservar como antecedentes históricos, debe definirse explícitamente esa política.

El notebook aborda esta responsabilidad en las operaciones de la sección 4.4. Al reservar una cita, crea también su comprobante. Al cancelar citas, actualiza los estados en ambas colecciones. Al eliminar una receta, modifica tanto la atención como su ficha. Estas operaciones utilizan transacciones para mantener coordinadas las escrituras.

### 4.5.3 Integridad y reglas de negocio

En MySQL, las claves primarias identifican los registros y las claves foráneas permiten comprobar que las relaciones apunten a registros existentes. En SaludUSM, las claves foráneas compuestas de `cita` comprueban que el médico esté asociado al centro seleccionado y a la especialidad indicada.

El esquema también define acciones ante eliminaciones. Por ejemplo, la relación de `receta` con `atencion` utiliza `ON DELETE CASCADE`, por lo que eliminar una atención elimina sus recetas relacionadas. La relación entre `cita` y `paciente` utiliza `ON DELETE SET NULL`, permitiendo conservar la cita cuando se elimina el paciente.

Además, se implementaron reglas mediante triggers y procedimientos. `FechaCita` rechaza la inserción de una cita en una fecha y hora pasadas. `ValidarAgendaInsert` llama a `ValidarBloqueCita` para comprobar que el médico y el paciente no tengan otra cita vigente en el mismo bloque. `ValidarAtencion` exige que la cita asociada esté en estado `Atendida`.

En MongoDB, `_id` identifica de manera única cada documento. Los campos de referencia, como `medico_rut` y `cita_id`, requieren lógica adicional para comprobar la existencia de los documentos relacionados. También es necesario programar qué ocurre con esas referencias al eliminar un documento.

En nuestra implementación, la función `reservar_cita` realiza en Python las comprobaciones de existencia del paciente, pertenencia del médico al centro y a la especialidad, fecha futura y disponibilidad del bloque. Su revisión temporal compara la fecha con el instante actual, por lo que no reproduce exactamente la comprobación conjunta de fecha y hora del trigger de MySQL.

MongoDB ofrece validación de esquema e índices únicos que pueden reforzar tipos de datos y restricciones. Sin embargo, las comprobaciones de relaciones implementadas en Python dependen de que las escrituras pasen por esas funciones. Por ejemplo, las atenciones migradas proceden de MySQL, pero una futura función para registrar atenciones directamente en MongoDB tendría que comprobar también el estado de la cita.

Las transacciones de la sección 4.4 mantienen unidas las modificaciones de las colecciones principales y sus copias. Por su parte, `write concern majority` solicita confirmación de la mayoría del replica set. La validación de reglas de negocio y la confirmación de replicación cumplen responsabilidades diferentes.

### 4.5.4 Lenguaje de consulta

La consulta de agenda diaria de un médico de la sección 4.3 permite comparar ambos lenguajes. Recibe el RUT del médico y una fecha, y devuelve sus citas ordenadas por hora con los datos del paciente, previsión, centro, especialidad y estado.

En SQL, `WHERE` filtra el médico y la fecha, los `JOIN` relacionan las tablas, `SELECT` determina las columnas del resultado y `ORDER BY` ordena las citas. En esta consulta se utilizan `LEFT JOIN` para conservar una cita aunque no tenga un paciente asociado.

En MongoDB, la función `agenda_medico` ejecuta un pipeline que utiliza `$match` para filtrar, `$sort` para ordenar y `$lookup` para incorporar documentos de las colecciones relacionadas. `$unwind` permite trabajar con los resultados de esos cruces y `$project` selecciona los campos que se mostrarán. Para conservar citas sin paciente, se utiliza `preserveNullAndEmptyArrays`.

Ambas consultas expresan la misma necesidad mediante estructuras diferentes. El notebook comprobó que devolvían las mismas tres filas para los parámetros seleccionados. Los filtros y cruces se ejecutan en los motores de base de datos, mientras que Python presenta y compara los resultados. Esta equivalencia confirma el resultado del ejemplo, pero no constituye una medición de rendimiento.

### 4.5.5 Flexibilidad del esquema

Si SaludUSM necesita registrar alergias, en el modelo relacional puede incorporarse un catálogo de alergias y una tabla intermedia que las relacione con los pacientes. Esta alternativa permite registrar varias alergias por paciente sin repetir sus descripciones.

Para los signos vitales, podrían agregarse columnas a una tabla relacionada con la atención. Si las mediciones varían considerablemente entre especialidades, también podría diseñarse una tabla de mediciones con tipo, valor y unidad. Ambas alternativas requieren definir la estructura y adaptar las consultas correspondientes.

En MongoDB, podría agregarse un arreglo `alergias` a `Pacientes` y un documento `signos_vitales` a `Atenciones`. Este último podría contener campos como temperatura, frecuencia cardíaca o presión arterial según la información disponible en cada atención. Los documentos anteriores podrían seguir existiendo sin esos campos.

Esta flexibilidad facilita la incorporación progresiva de información. Aun así, es necesario acordar nombres, tipos y unidades para que las consultas interpreten los datos correctamente. Si existe un validador de esquema, también debe ajustarse. MySQL permite igualmente trabajar con columnas JSON, por lo que la elección depende del diseño y de las restricciones que se quieran aplicar.

### 4.5.6 Comparación y recomendación para SaludUSM

| Aspecto | MySQL en SaludUSM | MongoDB en SaludUSM |
|---|---|---|
| Modelo | Tablas relacionadas mediante claves. | Documentos con referencias y arreglos embebidos. |
| Esquema | Columnas y tipos definidos en las tablas. | Campos variables, con posibilidad de validación. |
| Redundancia | La normalización reduce datos descriptivos repetidos. | Comprobantes y fichas contienen copias que deben mantenerse. |
| Integridad | Claves, restricciones, triggers y procedimientos. | `_id` único y comprobaciones de relaciones en la aplicación. |
| Consultas | SQL con `JOIN`, filtros y agrupaciones. | Pipelines con etapas como `$match`, `$lookup` y `$group`. |
| Cambios del modelo | Requieren adaptar tablas y relaciones. | Permiten agregar campos y documentos anidados progresivamente. |
| Operaciones coordinadas | Transacciones sobre las tablas involucradas. | Transacciones entre colecciones principales y copias. |

Para el **agendamiento**, recomendamos MySQL como fuente principal debido a las relaciones entre pacientes, médicos, centros y especialidades, y a las reglas que deben aplicarse al reservar una hora. En un sistema con

reservas simultáneas, también sería necesario revisar la concurrencia y reforzar la protección contra dobles reservas.

Para la **ficha clínica**, MongoDB resulta conveniente porque permite almacenar recetas y antecedentes variables dentro de documentos asociados a cada atención. La incorporación de signos vitales o información específica de una especialidad puede realizarse de manera progresiva, manteniendo criterios de validación.

Para el **panel de gestión**, MySQL es una alternativa adecuada si los datos operacionales se mantienen allí, ya que permite calcular indicadores mediante consultas SQL. MongoDB también puede resolverlos, como muestran las consultas de inasistencia y ranking del notebook. La selección debería considerar el origen de los datos y mediciones concretas de rendimiento.

Una arquitectura combinada podría utilizar MySQL para el agendamiento y MongoDB para la ficha clínica. Esta propuesta requiere definir qué motor es responsable de cada dato y cómo se sincronizan los identificadores y estados compartidos. La migración desarrollada en esta tarea demuestra el traspaso de información, mientras que una integración continua requeriría implementar esa sincronización.